Robustness Matters: Pre-Training Can Enhance the Performance of Encrypted Traffic Analysis

Luming Yang, Lin Liu, Jun-Jie Huang, *Member, IEEE*, Jiangyong Shi, Shaojing Fu, Yongjun Wang, and Jinshu Su, *Senior Member, IEEE*

***Abstract*—Models with large-scale parameters and pre-training have been leveraged for encrypted traffic analysis. However, existing researches primarily focused on accuracy, often overlooking the role of large-scale pre-trained parameters in enhancing robustness. While machine learning (ML) and deep learning (DL) models trained from scratch can achieve high accuracy, they exhibit limited robustness. When subjected to network noise in real-world, their identification results can fluctuate significantly, which is unacceptable. Unfortunately, current robustness evaluation methods neglect samples diversity and employ unreasonable noise settings. This field still lacks a reasonable quantitative description of models robustness. In this paper, we propose the PA-curve to display the distribution of sample's correct-decision stability, which can simultaneously reflect the model's accuracy and robustness. By calculating the area under the PA-curve, called PA-area, we enable the quantitative assessment of robustness for encrypted traffic analysis. Furthermore, we design a pre-trained model based on packet length sequence, and pre-trained it on TB-scale traffic. By fine-tuning on limited labeled training data, it can achieve downstream analysis tasks. We conduct experiments on five encrypted traffic datasets with different tasks. Besides accuracy, we analyzed the robustness of the pre-trained model and existing methods under common network disturbances, including packet loss, retransmission, and disorder. Experimental results demonstrated that, compared to ML-based and DL-based models trained from scratch, the pre-trained model can not only achieve high accuracy, but also exhibit greater resilience to network noise. The source code is available at https://github.com/Shangshu-LAB/BERT-ps**

***Index Terms*—Encrypted traffic analysis (ETA), robustness, pre-trained model, network noise.**

Received 3 March 2025; revised 15 August 2025; accepted 7 September 2025. Date of publication 24 September 2025; date of current version 9 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62472431, Grant 62201600, and Grant 62372462; and in part by the Science Research Plan Program through the National University of Defense Technology (NUDT) under Grant ZK22-50 and Grant ZK22-56. The associate editor coordinating the review of this article and approving it for publication was Dr. Yue Zhang. *(Corresponding author: Lin Liu.)*

Luming Yang is with the Academy of Military Science, Beijing 100091, China, and also with the National University of Defense Technology, Changsha 410073, China (e-mail: yang_lm108@nudt.edu.cn).

Lin Liu, Jun-Jie Huang, Jiangyong Shi, Shaojing Fu, and Yongjun Wang are with the College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: liulin16@nudt.edu.cn; jjhuang@nudt.edu.cn; shijiangyong@nudt.edu.cn; fusj_nudt@163.com; wwyyjj1971@126.com).

Jinshu Su is with the Academy of Military Science, Beijing 100091, China (e-mail: sjs@nudt.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3613970

I. Introduction

With the widespread adoption of encryption protocols such as TLS/SSL, the proportion of encrypted traffic is gradually increasing. Encryption is a double-edged sword: while it effectively protects user privacy, it poses significant challenges to network monitoring and supervision. Due to the ineffectiveness of port filtering and deep packet inspection (DPI), artificial intelligence (AI) driven analysis is now the mainstream of encrypted traffic analysis. Traditional machine learning (ML) methods [1], [2], [3], [4], [5], [6], [7], [8] rely on expert knowledge, while deep learning (DL) methods [9], [10], [11], [12] apply representation learning to extract features automatically. In recent years, our community has also proposed several pre-trained models [13], [14], [15], [16], [17], [18] for network traffic analysis. They undergo self-supervised pre-training on large amounts of unlabeled data to learn general knowledge, and then utilize a small amount of labeled data to learn task-specific knowledge by supervised fine-tuning (SFT). This approach can achieve high accuracy with limited training data. Researchers have been introducing more and more learnable parameters to enhance models' capabilities, thereby further improving encrypted traffic analysis accuracy.

Encrypted traffic analysis encompasses various tasks, including application identification, website fingerprinting, and malware detection, each varying in difficulty. Models with large-scale parameters often yield higher accuracy for more challenging tasks. However, ML-based, DL-based, and pre-trained models all yield satisfactory results in many cases. From an accuracy perspective, more complex pre-trained models may not be necessary.

Besides accuracy, model robustness is also critical yet often overlooked for encrypted traffic analysis. Network environment may undergo complex changes due to the joint effect of various factors [19], such as traffic burst [20], [21], traffic engineering [22], [23], partial network failures [24], [25], and network updates [26], [27]. Some methods can achieve high accuracy in specific network environments, but are not robust. As illustrated in Table I, when subjected to different levels of network noise, their identification results exhibit significant deterioration, which is unacceptable. We desire models that not only possess high accuracy, but also exhibit strong robustness to ensure consistent performance in diverse and noisy network environments.

In encrypted traffic analysis, some studies [28], [29], [30], [31], [32] only evaluate their model's robustness using accuracy under noise, and others [19], [33], [34], [35], [36],

TABLE I
THE IDENTIFICATION ACCURACY UNDER PACKET LOSS

Loss Rate (σ)	None	0.05	0.1	0.15	0.2	0.3
AppScanner [3]	0.9304	0.9293	0.9046	0.9013	0.8959	0.8806
ETC-PS [8]	0.9263	0.7819	0.7112	0.6881	0.6621	0.6435
FlowLens [5]	0.9239	0.9023	0.8792	0.8641	0.8271	0.7833
FS-Net [9]	0.9385	0.9280	0.8828	0.8709	0.8659	0.8275
GraphDApp [10]	0.7163	0.6858	0.6502	0.6106	0.5667	0.5804

* The network environment is affect with different packet loss rate (σ).
** The evaluation is on `DataCon2020` dataset.

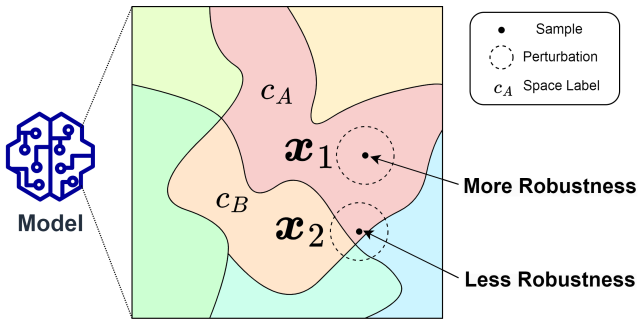


Fig. 1. The local robustness of samples varies depending on the location in the sample space. The model partitions the space into distinct decision regions, which are represented by different colors. For $\boldsymbol{x}_1$, which lies far from decision boundaries, its neighborhood remains entirely within one decision region (c_A), resulting in greater resistance to perturbations. For $\boldsymbol{x}_2$, which is close to decision boundaries, its neighborhood spans multiple decision regions (c_A, c_B, etc.), making the output more sensitive to perturbations.

[37] focus on improving model's robustness through techniques such as data augmentation. Unfortunately, they still lacks a reasonable quantitative description of model's robustness. Specifically, there are the following issues:

- **Neglecting Sample's Diversity.** Existing approaches [19], [31], [32], [38], [39], [40], [41], [42], [43] typically use the accuracy under noise to measure the model's robustness, without considering the differences across samples. For instance, as illustrated in Fig. 1, samples far from decision boundaries exhibit stronger robustness than those near the boundary. The model's robustness should reflect the overall distribution of local robustness, which cannot be only measured by the accuracy under noise.
- **Unreasonable Noise Settings.** Some existing works [28], [29], [30], [33], [34], [35], [36], [44], [45] only introduce perturbations in the feature space rather than the raw data during robustness evaluation. Different features are typically not distributed independently but exhibit correlations. Consequently, noise added to the feature space may not accurately reflect real-world perturbations of network flows. Artificial noise fails to accurately mimic perturbations in network flows, leading to unreasonable robustness evaluations that do not generalize to real-world.

Moreover, robustness evaluation metrics should also account for accuracy, as robustness without accuracy is meaningless.

In this paper, we propose a new robustness evaluation metric, called PA-curve, to display the distribution of sample's correct-decision stability and then calculated the PA-area value. They can capture the relationship between accuracy and identification probability difference within the sample space, thus providing a more comprehensive quantification of the model's robustness for encrypted traffic analysis. Specifically, for the PA-curve, the more right-side, the more robust; the more up-side, the more accurate. In addition, we also design a pre-trained model BERT-ps based on packet length sequence, and pre-trained it on TB-scale traffic. By fine-tuning on limited labeled training data, BERT-ps can achieve specific encrypted traffic analysis tasks, effectively. Experimental results demonstrate that BERT-ps not only achieves higher accuracy but also stronger resilience to network noise compared to ML-based models and DL-based models trained from scratch. These findings highlight the practical significance of exploring pre-trained models with large-scale learnable parameters for encrypted traffic analysis.

The key contributions of this paper are as follows:

1) For robustness analysis, we propose a new robustness evaluation metric called PA-curve. We can apply the PA-curve to show the local robustness distribution in the sample space with certain noise disturbances, and then calculate the area under the PA-curve named PA-area as an indicator to measure both the model's accuracy and its robustness.
2) To analyze the pre-training impact on robustness, we design a pre-trained model based on packet-length sequences for encrypted traffic analysis, named BERT-ps . By pre-training on massive unlabeled data, BERT-ps can learn relevant knowledge of network traffic, and achieve superior performances on specific analysis tasks through supervised fine-tuning.
3) We conduct extensive experiments on five public datasets. Compared to existing methods, BERT-ps with pre-trained parameters can not only achieve higher accuracy, but also have stronger robustness under network noise.

The remainder of this paper is organized as follows. We give a review of related work in Section II. In Section III, we introduce three perturbations and a novel metric for robustness. In section IV, we introduce the design details of BERT-ps . Then, we describes experimental setting in Section V and evaluate the effectiveness of the proposed approaches in section VI. Discussions are presented in Section VII. Finally, we concludes this paper in Section VIII.

II. RELATED WORK

As the development of encryption, there are many approaches for encrypted traffic analysis, which are mainly based on machine learning (ML) and deep learning (DL).

A. ML-Based Methods

These methods rely on expert knowledge to select artificial features and apply ML algorithms. Taylor et al. [2], [3] presented AppScanner, which only applies side-channel data to build fingerprints for Android apps. FlowLens [5] proposed by Barradas et al. collects features of packet distributions at line speed and classifies flows directly on the switches. Xu et al. [8] introduced path signature into encrypted traffic classification based on packet-length information. Anderson

and McGrew [1], [4] developed supervised machine learning models that take advantage of a unique and diverse set of encrypted network flow features including TLS handshake metadata, DNS contextual flows and HTTP contextual flows.

As for TLS/SSL encrypted traffic, TLS/SSL message sequence is also widely applied for identification [46], [47], [48], [49], [50]. Korczynski and Duda [46] first proposed Markov chain fingerprints for application traffic flows based on TLS/SSL message. On this basis, Shen et al. [47], [48] observed that the application attribute bigram contributes to application discrimination, and proposed an attribute-aware encrypted traffic classification method based on the second-order Markov Chains. Pan et al. [49] proposed an HTTPS application classification method using weighted ensemble learning, named WECN. Liu et al. [50] proposed the MaMPF for encrypted traffic classification which makes use of length block sequence (LBS) from the power-law division of packet-length sequence and the relative probability of all applications. But these methods cannot analyze other types of encrypted traffic except TLS/SSL.

B. DL-Based Methods

These methods automatically extract features from raw network traffic by representation learning. Liu et al. [9] designed an RNN-based end-to-end Flow Sequence Network (FS-Net), which jointly learns the representative features from the raw flow sequences and classifies them together. Shen et al. [10] constructed Traffic Interaction Graph (TIG) as an representation based on the packet-length and packet direction.

To leverage large-scale unlabeled data for traffic representations, several pre-trained models [13], [14], [15], [16], [17], [18] have been proposed in recent years. PERT proposed by He et al. [13] and ET-BERT proposed by Lin et al. [14] learn deep contextual datagram-level representations from unlabeled traffic data by masked language modeling. Inspired by computer vision, Flow-MAE [16] and YaTC [15] utilized the masked auto-encoder (MAE) pre-training approach to encode network flows from byte matrices Zhou et al. [18] proposed TrafficFormer, which introduces a fine-grained multi-classification task to enhance its representation capabilities in the pre-training stage. Instead of Transformer, NetMamba [17] applied Mamba architecture for comprehensive traffic representation to address efficiency issues.

C. Robustness of Traffic Analysis

Beyond accuracy, robustness has become an increasingly important consideration in network traffic analysis. A common approach for robustness testing involves introducing random perturbations in the feature space, such as Gaussian noise [30] or Laplacian noise [29]. Similarly, studies on adversarial robustness [34], [35], [36] often focus on perturbations within the feature space. Wang et al. [44] proposed BARS, a general robustness certification framework for deep-learning-based traffic analysis systems based on boundary-adaptive randomized smoothing. RBLJAN [37] assesses robustness by randomly inserting bytes into packets. However, such artificial noise differs significantly from real-world perturbations, potentially misleading robustness evaluations.

Several studies have evaluated model's performance in different network environments. In robustness testing, Zhao et al. [31], [32] introduced packet loss, retransmission, and disordering, while [43] incorporates transmission delay. Moreover, [39], [40], [41], [42] assess analytical performance under varying traffic replay rates. After investigating real diverse network environments, Rosetta [19] leverages TCP-aware traffic augmentation mechanisms and self-supervised learning to understand implicit TCP semantics, and hence extracts robust features of TLS flows. These studies primarily use accuracy under network noise as the metric, which does not account for local robustness difference in sample space.

Additionally, other research explores robustness in network traffic analysis concerning concept drift [32], [33], [51], [52], adversarial samples [28], [34], [35], [36], [45], and label noise [53].

III. Revisiting Robustness Evaluation

In this section, we first introduce three common types of network disturbances subjected to network flows, and then introduce new comprehensive metrics, PA-curve and PA-area, for measuring model's robustness.

We define a network flow with n packets as $\boldsymbol{x} = \langle \mathcal{P}_1, \mathcal{P}_2, \cdots, \mathcal{P}_n \rangle$, where $\mathcal{P}_i$ is the i-th packet. The identification model for encrypted traffic analysis can be defined as a function $f : \mathcal{X} \mapsto \mathcal{Y}$ where $\mathcal{X}$ is the sample space and $\mathcal{Y} = \{c_1, c_2, \cdots, c_m\}$ is the label space.

A. Noise Impact on Network Flows

Network environment may undergo complex changes due to the joint effect of various factors [19], such as traffic burst [20], [21], traffic engineering [22], [23], partial network failures [24], [25], and network updates [26], [27], etc. Network noise is ubiquitous in real-world network environments, particularly under conditions of complex network topologies and high traffic loads. Encrypted traffic analysis is typically deployed in real-world network environments, making it essential to account for the impact of noise on analysis results. We define $\boldsymbol{\varepsilon}$ is a specific perturbation from the network noise $\Upsilon(\sigma)$ with noise parameter σ, i.e. $\boldsymbol{\varepsilon} \sim \Upsilon(\sigma)$. For a network flow $\boldsymbol{x}$ and a perturbation $\boldsymbol{\varepsilon}$, the disturbed sample is expressed as $\tilde{\boldsymbol{x}} = \boldsymbol{x} \oplus \boldsymbol{\varepsilon}$ where $\oplus$ indicates the perturbation addition operations.

The transmission patterns of network flows are susceptible to network noise, which can result in packet loss, retransmission, and disorder. These phenomena introduce perturbations to the network flow data (such as packet-length sequences). In detail, there are three common impacts of network noise on network flows, which are defined as follows:

- **Packet Loss:** Due to the performance limitations of network device, some packets may not be captured, leading to missing packets in the collected network flow. For packet loss perturbations, we assume that each packet is lost with a probability of σ. We defined the perturbation of packet loss as $\Upsilon_{\text{loss}}(\sigma)$.
- **Packet Retransmission:** There are many types of error during TCP transmission, such as corrupted packets, timeout, etc. If the sender does not receive an acknowledgment

(ACK) from the receiver before the re-transmission time-out, it will re-transmit the packet to ensure reliable transmission. For packet re-transmission perturbations, we assume that each packet is re-transmitted with a probability of σ. We defined the perturbation of packet re-transmission as $\Upsilon_{\text{retrans}}(\sigma)$.

- **Packet Disordering:** Due to network environment fluctuations, the order in which packets are sent and received may be inconsistent. The packets of captured network flows may be out of order. For packet disorder perturbations, we assume that each packet has a probability of σ swapping with the next packet. We defined the perturbations of packet disorder as $\Upsilon_{\text{disorder}}(\sigma)$.

Due to the error correction and check mechanisms inherent in network protocols, network noise typically does not affect the content of the packets. Therefore, this study focuses solely on the perturbation introduced by network noise to packet transmission patterns in network flows.

In addition, packet loss, retransmission, and disorder are three common impacts of network noise on packet-length sequences. They are widely employed as network noise settings for robustness evaluations in existing studies [19], [31], [43]. Our work aims to establish a quantitative and systematic robustness evaluation metric for encrypted traffic analysis. As a result, it requires relatively standardized and interpretable noise settings.

B. Local Robustness in Sample Space

Robustness should reflects the ability of a system to resist environment change without adapting its initial configuration [54]. As for traffic analysis, the robustness refers to whether the analysis model can stably output correct results when the network flow sample is perturbed by network noise. In addition, we consider the robustness of the model only if it can achieve acceptable accuracy. If the model has a poor identification performance, the robustness is meaningless.

Essentially, the analysis model divides the sample space into different regions, each corresponding to a class. For a trained model, the spatial structure of these regions is typically complex. For network flow samples, some are far from the region boundaries, while others are closer. Samples that are farther from region boundaries tend to be more robust. Robustness can be reflected by the probability that the model can stably output correctly in the sample's neighborhood. Therefore, inspired by certified robustness [55], we can measure the robustness of a sample by analyzing the distribution of different classes within its neighborhood, i.e., the probability distribution of perturbed samples falling into different class regions.

For a network flow $\boldsymbol{x} \in \mathcal{X}$ with class $c \in \mathcal{Y}$, the model judges within its neighborhood to be a category y_i with the confidence probability p_i. Among them, we set the classes with the highest and second-highest confidence probabilities as c_A and c_B corresponding to p_A and p_B, respectively. As for $\boldsymbol{x}$ with the perturbation $\Upsilon(\sigma)$, c_A and c_B can be expressed as follows:

$$\begin{aligned} c_A &= \arg\max_{c\in\mathcal{Y}} \mathbb{P}\{f(\boldsymbol{x}\oplus\boldsymbol{\varepsilon}) = c | \boldsymbol{\varepsilon} \sim \Upsilon(\sigma)\}, \\ c_B &= \arg\max_{c\in\mathcal{Y}\setminus\{c_A\}} \mathbb{P}\{f(\boldsymbol{x}\oplus\boldsymbol{\varepsilon}) = c | \boldsymbol{\varepsilon} \sim \Upsilon(\sigma)\}. \end{aligned} \tag{1}$$

As long as $p_A > p_B$, it indicates that perturbations within the neighborhood can produce the correct output. In addition, the greater the difference between p_A and p_B, the more stable the output will be. Therefore, the robustness of the sample can be measured by $\Delta p = p_A - p_B$.

However, it is impossible to exactly evaluate the region boundaries of sample space or to exactly compute p_A and p_B for a neural-network-based classifier [55]. Instead, we estimate Δp based on Monte-Carlo method, which is guaranteed to succeed with arbitrarily high probability. To ensure the reliability, in practice, we apply the binomial proportion confidence interval [56] to calculate the lower bound of p_A and the upper bound of p_B, i.e., $\underline{p_A}$ and $\overline{p_B}$.

We apply Clopper-Pearson method [57] to estimate the proportion confidence interval. When only the experiment number n and the success number k are known, we estimate the lower and upper bound ($\underline{p}$ and $\overline{p}$) of a success probability $p = \frac{k}{n}$ as follows:

$$F_\beta^{-1}\left(\frac{\alpha}{2}; n-k+1, k\right) \le p \le F_\beta^{-1}\left(1-\frac{\alpha}{2}; k+1, n-k\right) \tag{2}$$

where $F_\beta^{-1}(\cdot)$ is the inverse cumulative distribution function (ICDF) of the beta distribution. More detail about the principle of the binomial proportion confidence interval base on Clopper-Pearson method is illustrated in Appendix A.

Algorithm 1 Calculation of $\Delta\hat{p}$ Based on Monte Carlo Method

Require: The sample $\boldsymbol{x}$ and the network noise $\Upsilon(\sigma)$;
The Monte Carlo sample number N;
The confidence coefficient $1-\alpha$;
Ensure: $\Delta\hat{p}$
$counts = \{c : 0 | c \in \mathcal{Y}\}$;
for $i = 1$ **to** N **do**
  $c' \leftarrow f(\boldsymbol{x}\oplus\boldsymbol{\varepsilon})$ where $\boldsymbol{\varepsilon} \sim \Upsilon(\sigma)$;
  $counts[c']$++;
end for
$c_A, c_B \leftarrow$ top two index in $counts$;
$n_A, n_B \leftarrow counts[c_A], counts[c_B]$;
$\underline{p_A} \leftarrow$ `LowerConfBound`$(n_A, N, 1-\alpha)$;
$\overline{p_B} \leftarrow$ `UpperConfBound`$(n_B, N, 1-\alpha)$; **return** $\underline{p_A} - \overline{p_B}$

Therefore, we use $\Delta\hat{p} = \underline{p_A} - \overline{p_B}$ to measure the local robustness of samples, which is illustrated in Algorithm 1.

C. PA-Curve and PA-Area

Since the robustness varies among different samples, the global robustness of the model should be based on the distribution of local robustness. It makes no sense to consider the local robustness of samples that are incorrectly predicted by the model, so the models robustness should also take accuracy into account.

1) PA-Curve: It is designed to show the distribution of the local robustness on the sample space under a certain network noise perturbation. For the encrypted traffic analysis model, PA-curve can reflect the relationship between identification **P**robability differences and **A**ccuracy within the sample space. Specifically, a point (x, y) on the PA-curve represents the proportion of samples that can be correctly predicted by the

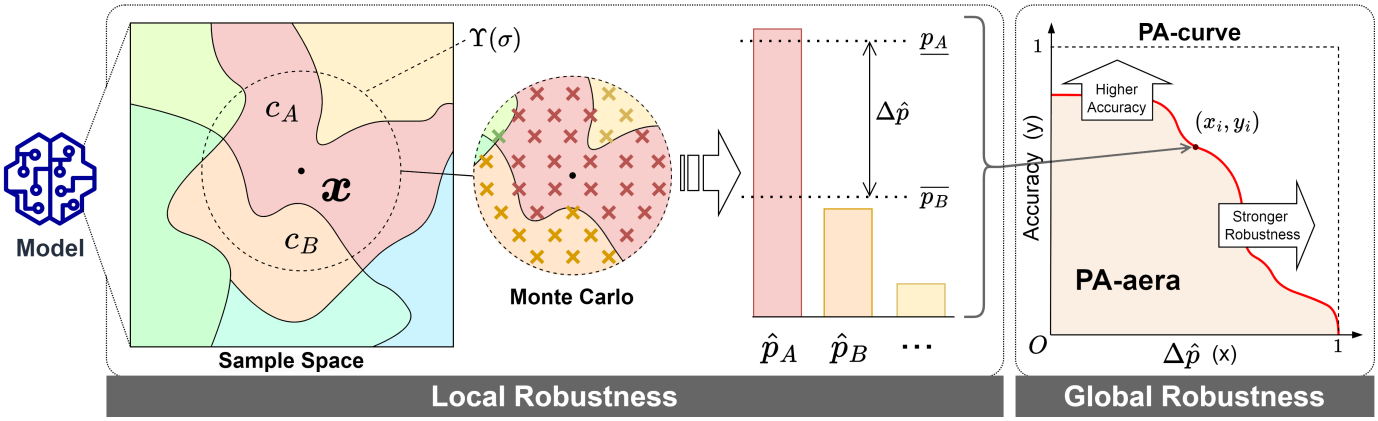


Fig. 2. Within the region defined by the neighborhood of $\boldsymbol{x}$ under perturbation $\Upsilon(\sigma)$, the classes with the highest and second-highest confidence probabilities are c_A and c_B, respectively. (This figure is just a schematic, and $\boldsymbol{x}$ is not in Euclidean space.) The confidence probabilities can be respectively estimated as $\hat{p}_A$ ($\underline{p_A}$) and $\hat{p}_B$ ($\overline{p_B}$) using Monte Carlo method. For a test set, the PA-curve is the distribution of $\Delta\hat{p}$ within different accuracy. PA-area is the area under the PA-curve.

model when the identification probability difference is at least x. The equation of PA-curve can be mathematically expressed as follows:

$$y = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\left\{c_A^{(i)} = c^{(i)} \wedge \left(\underline{p_A^{(i)}} - \overline{p_B^{(i)}}\right) \geq x\right\}, \tag{3}$$

where $\mathbb{I}(\cdot)$ is the indicator function, $c^{(i)}$ is the label of the i-th sample $\boldsymbol{x}^{(i)}$ in the test set $\{\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \cdots, \boldsymbol{x}^{(N)}\}$. As illustrated in Fig. 2, for the PA-curve, the more right-side, the more robust; the more up-side, the more accurate. Therefore, PA-curve can measure both accuracy and robustness.

Note: *As illustrated in Eq. 3, for the PA-curve, a higher position indicates that more samples identified by the model correctly ($c_A^{(i)} = c^{(i)}$) under the same constraints on identification probability differences ($\Delta\hat{p}^{(i)} = \underline{p_A^{(i)}} - \overline{p_B^{(i)}} \geq x$), indicating higher accuracy. Additionally, a rightward shift indicates that the model exhibits a larger identification probability difference ($\Delta\hat{p}^{(i)}$) of samples with a certain accuracy ($Acc = \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\{c_A^{(i)} = c^{(i)}\}$), indicating greater robustness.*

2) PA-Area: The area under the PA-curve, called PA-area, can serve as a comprehensive metric for measuring both robustness and accuracy. As shown in Fig. 2, the upper and more right PA-curve lead to the higher PA-area. PA-area can be calculated as follows:

$$\text{PA-area} = \int_0^1 y\mathrm{d}x = \frac{\sum_{i=1}^{N-1}(x_{i+1} - x_i)(y_{i+1} + y_i)}{2(N-1)}, \tag{4}$$

where (x_i, y_i) is the i-th point of PA-curve corresponding to the sample $\boldsymbol{x}^{(i)}$ in the test set. ($0 \leq x_1 \leq \cdots \leq x_N \leq 1$, $1 \geq y_1 \geq \cdots \geq y_N \geq 0$.) Consequently, PA-area is a reasonable quantitative description of model's robustness for network traffic analysis.

Note: *As the perturbation parameter σ approaches zero, the neighborhood of a sample point corresponds to a single label, meaning $p_A \to 1$ and $p_B \to 0$. In this case, the PA-area reduces to the accuracy. The detailed proof is as follows:*

$$\begin{aligned}
\text{PA-area} &= \int_0^1 \lim_{\sigma\to 0} y\left(x; \boldsymbol{\varepsilon} \sim \Upsilon(\sigma)\right)\mathrm{d}x \\
&= \int_0^1 \frac{1}{N}\sum_{i=1}^{N}\lim_{\sigma\to 0}\mathbb{I}\left\{c_A^{(i)} = c^{(i)} \wedge \left(\underline{p_A^{(i)}} - \overline{p_B^{(i)}}\right) \geq x | \boldsymbol{\varepsilon} \sim \Upsilon(\sigma)\right\}\mathrm{d}x \\
&= \int_0^1 \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\{c_A^{(i)} = c^{(i)} \wedge \underbrace{1 \geq x}_{\text{True}}\}\mathrm{d}x \\
&= \frac{1}{N}\sum_{i=1}^{N}\mathbb{I}\{c_A^{(i)} = c^{(i)}\} = Acc
\end{aligned}$$

IV. Design Details of BERT-Ps

Packet-length sequences are widely used in encrypted traffic analysis due to their universality and low cost, but susceptible to network noise. To demonstrate that pre-training can enhance the model's robustness, it is essential to compare pre-trained models with traditional ML-based and DL-based models. Nevertheless, no pre-trained model based on packet-length sequences has been proposed for encrypted traffic analysis. Therefore, we design a pre-trained packet-length sequence model named BERT-ps , which is illustrated in Fig. 3.

A. Tokenization

In this module, we convert network flows into token sequences of packet length sequences. It transforms network flows into a format that can be directly fed into the model.

First, we record the captured network flows in the form of a packet-length sequences. A network flow is defined by a five-tuple (srcIP, srcport, dstIP, dstport, and protocol), i.e., a collection of bidirectional packets between the client (srcIP, srcPort) and the server (dstIP, dstPort). To avoid the influence of functional packets (e.g., SYN packets and ACK packets) within the network flow, we ignore packets without transport layer payload. The absolute value of the packet-length is the packet transport layer load length, while the sign of the packet length indicates the direction of the packet.

We apply packet length sequence to represent the transmission pattern of network flows. To mitigate the adverse impact of functional packets (e.g., SYN packets and ACK packets),

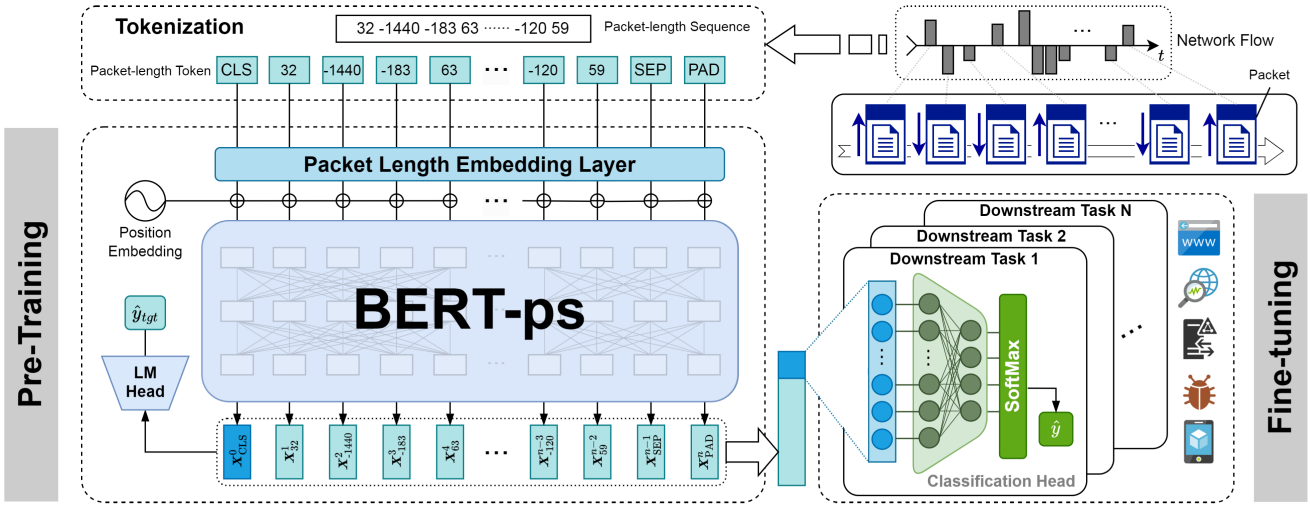


Fig. 3. The overview of BERT-ps for encrypted traffic analysis.

packets without transport layer payload are excluded in the packet length sequence. For the packet length, its absolute value is the length of the transport layer payload, while its sign indicates the direction of the packet in the network flow. We define that the uplink packet length is positive, while the downlink packet length is negative. For the tokenization of packet length sequences, we utilize packet lengths with direction directly as tokens.

We introduce 5 special tokens (`[CLS]`, `[SEP]`, `[PAD]`, `[MASK]`, and `[UNK]`). `[CLS]` and `[SEP]` are prefixed to indicate the beginning and the end of the sequence, respectively. For fixed-length input, sequences with insufficient length are padded with `[PAD]`, which are subsequently masked within the model to prevent them from affecting the representation of overall sequence. `[MASK]` and `[UNK]` are applied for replacing masked tokens and unknown tokens.

B. Model Structure

There are 3 modules in BERT-ps , including packet length embedding, transformer encoder, and classification head.

1) Packet-Length Embeddin: Each packet-length token is map into Euclidean space to represent the semantic information as a fixed-length vector. The packet-length embedding is composed of token embedding and position embedding.

The vocabulary includes packet-lengths ranging from 1 to 1500 with 2 directions and 5 special tokens. For network flows, tokens with same packet length have different semantic meanings depending on its position. So we apply the position encoding to each token. Both token and position embedding are achieved by embedding layers, and then combined before being fed into BERT-ps . Consequently, the embedding result of the packet-length token at position i is expressed as follows:

$$\boldsymbol{E}^i_{\text{byte}} = \boldsymbol{E}_{\text{pl}} + \boldsymbol{E}^i_{\text{pos}}, \tag{5}$$

where its packet length embedding and positional encoding are represented as $\boldsymbol{e}_{\text{pl}}$ and $\boldsymbol{e}^i_{\text{pos}}$, respectively.

2) Transformer Encoder: BERT-ps is developed based on Bi-directional Transformer Encoder to represent packet length sequences of encrypted network flows. Given that encrypted traffic analysis prioritizes the understanding of network flows rather than generation, BERT [58] (Encoder-Only architecture) is more suitable rather than Decoder-Only models (such as GPT [59]). In addition, the output vector corresponding to `[CLS]` is regarded as the representation of the network flow.

3) Classification Head: It is the module within BERT-ps that maps the learned feature representations to class predictions for specific downstream tasks. The classification head consists of a fully-connected layer followed by a Softmax layer. We input the representation into the classification head to produce the identification result, which is expressed as follows:

$$\boldsymbol{p} = \text{Softmax}\left(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}\right), \tag{6}$$

where $\boldsymbol{p} = \left[p_c\right]_{c\in\mathcal{Y}}$ is the probability vector that indicates the network flow belongs to different categories in $\mathcal{Y}$. $\boldsymbol{W}$ and $\boldsymbol{b}$ are the weight and bias of the fully connected layer, respectively.

C. Pre-Training and Fine-Tuning

The BERT-ps is pre-trained on packet length sequences of TB-scale traffic. After supervised fine-tuning (SFT) on limited labeled training data, it can achieve downstream analysis tasks.

1) Pre-Training: The goal of pre-training on large-scale unlabeled packet-length sequences is to capture the underlying structure and common patterns by self-supervised learning, thus acquiring a general and meaningful representation of network flows. We utilize Masked Language Modeling (MLM) for pre-training. The core idea is to mask certain tokens (`[MASK]`), and then predict their original values based on the surrounding unmasked tokens within the context. The Language Model (LM) head consists of a fully-connected layer with softmax, which is applied to predict masked tokens.

For the input sequence X with k masked tokens, the loss function of the pre-training process is defined as follows:

$$\mathcal{L}_{\text{MLM}} = -\sum_{i=1}^{k} \log \mathbb{P}\{\text{MASK}_i = \text{token}_i|\tilde{X};\Theta\}, \tag{7}$$

where Θ is the model's parameters, $\tilde{X}$ is the masked input data and MASK_i is the masked token at the i-th position.

During pre-training, 15% tokens in the input sequence are masked, which is similar to the pre-training of BERT [58]. For masked 15% tokens, three operations are applied: 80% are directly replaced with [MASK], 10% are replaced with new tokens, and the other 10% remain unchanged.

2) Fine-Tuning: Although the pre-trained model can generate the versatile and meaningful representations of packet-length sequences, it cannot solve the problem end-to-end. Specifically, the output of the pre-trained model serves as the representation vectors for the packet-length sequence, rather than the final classification result. In order to enable end-to-end analysis, it is required to append a classification head on the pre-trained model and pass representation results to the classification head to produce the final decision. During the training process, the pre-trained parameters will be fine-tuned to better adapt the model to specific downstream tasks.

During the fine-tuning process, the loss function is set to cross entropy loss, which is defined as follows:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c\in\mathcal{Y}} y_c^{(i)} \log p_c^{(i)}, \tag{8}$$

where $y_c^{(i)}$ is the ground truth label, and $p_c^{(i)}$ is the predicted probability that the i-th sample belongs to class c.

After pre-training, the model's parameters generally reach a stable state. But the parameters in the classification head are without training. Fine-tuning all model's parameters on a small-scale training set could cause unintended changes to stable pre-trained parameters due to updates of untrained parameters, potentially leading to over-fitting. Therefore, we warm-up the gradients of the classification head with all the pre-trained parameters frozen before full-parameter fine-tuning, enabling the model to achieve improved performance.

V. Experimental Settings

In this section, we describe implementation, pre-training data, dataset, and baselines in detail.

A. Implementation

Our experiments are performed on a server, which is equipped with the following configuration: AMD EPYC(TM) 7542 CPU @ 2.90GHz, 8× NVIDA GeForce RTX 6000 Ada, 512GB of RAM. In following experiments, all the neural network structures are implemented by PyTorch (version 2.2.1).

We adopt the hyper-parameter settings of BERT-base [58] as a reference for BERT-ps , which is illustrated in Table II.

As for the evaluation of model's robustness (Algorithm 1), we set the Monte Carlo sample number N as 1000, and set the confidence coefficient as 0.999, i.e., $\alpha = 0.001$.

B. Pre-Training Data

We collected a large amount of network traffic (corresponding about 12.1TB pcap files) at a network gateway within a week. After data processing, a total of 76.11M packet-length sequences were generated (about 2.96B tokens). Referring to the pre-training data amount of BERT [58], this amount of data is enough to support the pre-training of BERT-ps .

TABLE II
The Hyper-Parameter Setting of BERT-ps

Hyper-Parameter	Value	Hyper-Parameter	Value
H	768	Epochs	5
L	12	Steps	7.5M
N_{head}	12	Batch Size	512
d_{FF}	3072	Learning Rate	2e-5
Vocabulary size	3005*	Optimizer	AdamW [60]
Max length	256		

* It contains 1-1500 packet length with 2 directions and 5 special tokens ([CLS], [UNK], [SEP], [MASK], and [PAD]).
** H, L, N_{head}, and d_{FF} refer to hidden size, hidden layer number, attention head number, and intermediate size of feed forward layer.

In experiments, all network traffic data were split and parsed based on Zeek.[1] We developed a Zeek plugin to record the packet-length sequences for each network flow. Considering that "mouse flows" are too short to provide meaningful information on packet-length sequences, we discarded flows with fewer than 5 packets with non-zero transport layer payload.

C. Datasets

To evaluate the performance of BERT-ps and existing baselines, we conduct experiments on five public datasets with different tasks for encrypted traffic analysis, including DataCon2020 [61], DataCon2021-p1 [62], DataCon2021-p2 [62], EBSNN-dataset [11], and CSTNET-TLS1.3 [63].

- **DataCon2020:** [61] It is a malware encrypted traffic dataset collected by QI-ANXIN Technology Research Institute. This dataset is derived from the TLS/SSL encrypted traffic generated by malware and normal software (both are exe types) in the Tianqiong sandbox.
- **DataCon2021-p1:** [62] It is an encrypted proxy traffic classification dataset, which is generated by accessing a list of websites using Selenium automatically. After data cleaning, the dataset contains encrypted network flows from 6 different proxy software.
- **DataCon2021-p2:** [62] It is an encrypted-proxy-based website traffic identification dataset. The traffic was generated by accessing different websites through the same encryption proxy software. After data cleaning, the dataset contains network flows generated from 22 websites.
- **EBSNN-dataset:** [11] It includes network traffic generated from using applications and accessing websites. After data cleaning, the dataset contains 22 categories.
- **CSTNET-TLS1.3:** [63] This traffic dataset are acquired from Alexa Top-5000 deployed with TLS 1.3, which is collected on CSTNET by [14]. After data cleaning, we select 80 application from the original dataset.

All these datasets were public after 2020, and better aligned with the current network environment. Raw network traffic data are parsed by Zeek plugins we developed as well. We also remove any flows containing fewer than 5 packets with payload, followed by removing any classes with fewer flows.

[1] https://zeek.org

D. Baselines

We use five existing packet-length-sequence-based works as baselines, including AppScanner [3], ETC-PS [8], FlowLens [5], FS-Net [9], and GraphDApp [10]. The basic algorithms of these baselines they applied include machine learning and deep learning. In detail, these existing works are as follows:

- **AppScanner:** [3] It is a ML-based method based on Random Forest to identify smartphone applications from encrypted traffic. It extracts statistical features from uplink flow, downlink flow, and complete flow as the feature set, where each flow provides several statistical features (i.e., minimum, maximum, mean, standard deviation, etc).
- **ETC-PS:** [8] It is a ML-based encrypted traffic classification method with path signature. The multi-scale path signature of packet-length-sequence-based traffic path is computed as the distinctive feature to train the traditional machine learning classifier, such as Random Forest.
- **FlowLens:** [5] It uses sampled packet-length distribution of network flows as features to train the identification model by supervised learning, i.e., Random Forest. We use the hyperparameter setting with the best accuracy used in the paper to retrain the model.
- **FS-Net:** [9] It is a DL-based encrypted traffic analysis method, which use packet-length sequence information for supervised learning. It leveraged reconstruction loss to ensure that the extracted features by Bi-GRU contain all the information of the flow sequences.
- **GraphDApp:** [10] It is a DL-based method based on Graph Neural Network (GNN). It constructed Traffic Interaction Graph (TIG) to represent network flows based on the packet-length and packet direction, and then applied GNN-based supervised learning.

In addition, we also applied four pre-trained models, including ET-BERT [14], YaTC [15], NetMamba [17], and Trafficformer [18] as baselines for accuracy analysis. The input to these pre-trained models is payload byte streams rather than packet length sequences. Due to possible training biases about network settings, packet headers are excluded from the input.

VI. Evaluation

In this section, we conduct a set of experiments to evaluate the performance of the pre-trained BERT-ps and existing baselines, including accuracy and robustness.

A. Identification Accuracy Analysis

The accuracy serves as the primary performance metric for network traffic analysis. We evaluated the accuracy of BERT-ps on five public datasets and compared it with various baselines. Metrics about accuracy we applied are detailed in Appendix B. Experimental results are presented in Table III.

1) Higher Accuracy Than Baselines: Although the task and complexity of datasets vary, Table III demonstrates that BERT-ps outperforms packet-length-sequence-based baselines on all datasets, and surpasses all existing baselines on three datasets. Notably, it achieves approximately 7% and 5% higher accuracy compared to the current state-of-the-art(SOTA) methods on `DataCon2021-p2` and `CSNET-TLS1.3`, respectively. Furthermore, on `DataCon2020` and `EBSNN-dataset`, BERT-ps performance is only marginally lower, by approximately 1.2% and 2.5%, than the best pre-trained models. Experimental results substantiate the adaptability of BERT-ps to diverse encrypted traffic analysis tasks.

2) Pre-Training Enhance Accuracy: As shown in Table III, the model with pre-trained parameters and supervised fine-tuning (SFT) performs consistently better than the model trained from scratch (TFS) across all encrypted traffic analysis tasks in terms of accuracy. This suggests that the pre-trained parameters can effectively enhance the model's performance. However, the impact of pre-training on model performance varies across different tasks. For `DataCon2021-p2`, which contains 22 classes, the accuracy and F_1 scores are both improved about 2.8%. In contrast, for the binary classification task of `DataCon2020`, the improvement is less than 0.8%.

3) Packet-Length Sequence Is More General: Existing pre-trained models are primarily based on raw byte streams. As shown in Table III, we also compare BERT-ps with these pre-training-based approaches. While existing pre-trained models can achieve slightly higher accuracy than BERT-ps on `DataCon2020` and `EBSNN-dataset`, their performance is noticeably weaker on the other three tasks. On the website identification task under encrypted tunnels (`DataCon2021-p2`), existing pre-trained models completely fail, whereas BERT-ps still achieves an accuracy of 90%. On the encrypted proxy classification task (`DataCon2021-p1`), packet-length-sequence-based methods significantly outperform pre-trained models. Compared to `DataCon2020` and `EBSNN-dataset`, which only contain encrypted traffic with outdated TLS versions, BERT-ps demonstrates a clear advantage on the TLS 1.3 website identification task (`CSTNET-TLS1.3`), achieving an accuracy of 97.5%. Experimental results indicate that the available byte-pattern in packet content diminish as the encryption degree increases, leading to the failure of approaches based on raw byte streams. Consequently, methods based on packet sequence length are more general for encrypted traffic analysis.

B. Robustness Analysis

We then conduct a detailed analysis of model's robustness using this dataset as a case study. We applied PA-curve and PA-area as metrics for robustness analysis. Specifically, we evaluate the impact of three common types of perturbation: packet loss, packet re-transmission, and packet disorder. These perturbations affect packet-length sequences but not packet content due to the checksum and error correction in protocols. Existing pre-trained models [14], [15], [17], [18] are not suitable for the robustness analysis in this subsection. As a result, we only compare BERT-ps with packet-length-sequence-based methods, including AppScanner [3], ETC-PS [8], FlowLens [5], FS-Net [9], and GraphDApp [10].

On `DataCon2020`, both BERT-ps and existing baselines can achieve relatively high accuracy. Figure 4 illustrates the PA-curve of the model under varying perturbation parameters.

TABLE III

THE IDENTIFICATION RESULTS OF BERT-PS AND BASELINES ON FIVE PUBLIC DATASET

Methods		DataCon2020 [61]		DataCon2021-p1 [62]		DataCon2021-p2 [62]		EBSNN-dataset [11]		CSTNET-TLS1.3 [63]	
		Accuracy	macro-F_1	Accuracy	macro-F_1	Accuracy	macro-F_1	Accuracy	macro-F_1	Accuracy	macro-F_1
AppScanner [3]		0.9304	0.9284	0.9218	0.9133	0.8321	0.8298	0.8663	0.8481	0.8451	0.8369
ETC-PS [8]		0.9263	0.9242	0.8588	0.8311	0.8153	0.8131	0.7958	0.7686	0.7764	0.7679
FlowLens [5]		0.9239	0.9217	0.9391	0.9330	0.7971	0.7936	0.8520	0.8288	0.9114	0.9068
FS-Net [9]		0.9385	0.9362	0.9401	0.9356	0.8298	0.8243	0.8607	0.8369	0.9208	0.9160
GraphDApp [10]		0.7163	0.7159	0.8019	0.7958	0.3826	0.3509	0.5020	0.4096	0.6236	0.6207
ET-BERT [14]		0.9583	0.9567	0.8676	0.7698	0.0439	0.0363	0.9213	0.9152	0.8839	0.8748
YaTC [15]		0.9562	0.9563	0.7302	0.6946	0.0574	0.0533	0.8795	0.8835	0.7026	0.6941
NetMamba [17]		**0.9616**	**0.9600**	0.8107	0.7598	0.0325	0.0029	0.8523	0.8405	0.8394	0.8267
TrafficFormer [18]		0.9537	0.9522	0.8577	0.7617	0.0298	0.0091	**0.9492**	**0.9382**	0.9144	0.9073
BERT-ps	TFS	0.9414	0.9392	0.9567	0.9495	0.8757	0.8709	0.9048	0.8869	0.9598	0.9573
	SFT	**0.9491**	**0.9472**	**0.9694**	**0.9642**	**0.9034**	**0.8993**	**0.9260**	**0.9115**	**0.9748**	**0.9731**

* SFT refers to the model trained by Supervised Fine-Tuning with pre-trained parameters, while TFS denotes the model Trained From Scratch.

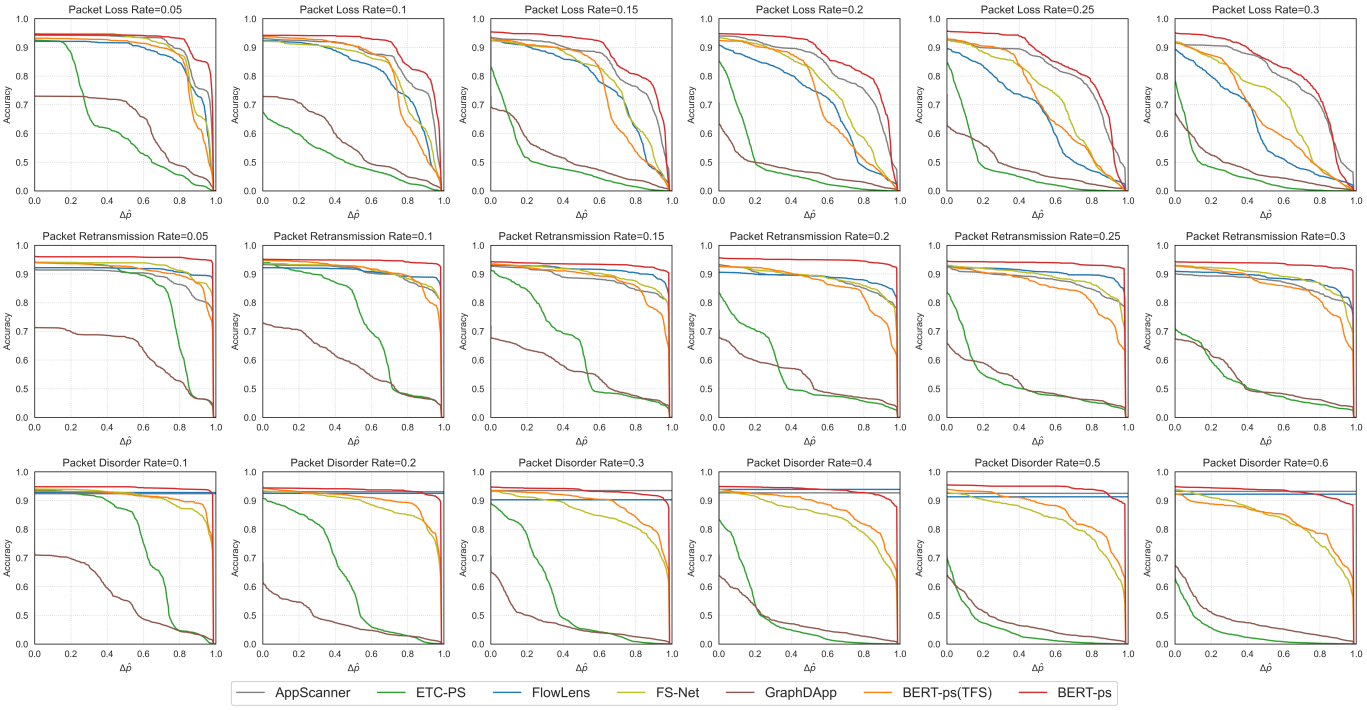


Fig. 4. The PA-curve of BERT-ps and baselines under different types and intensities of network noise on DataCon2020. Each row in the figure corresponds to a type of network noise, i.e., packet loss, packet retransmission, and packet disorder.

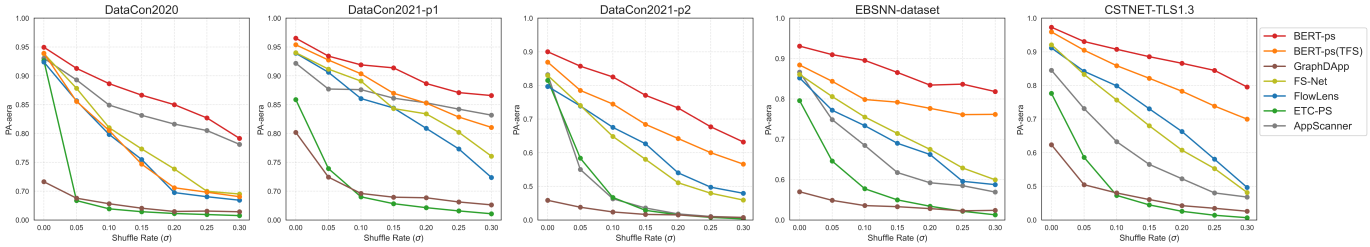
TABLE IV

THE PA-AREA RESULTS AT DIFFERENT INTENSITIES OF NETWORK NOISE ON DATACON2020ETA

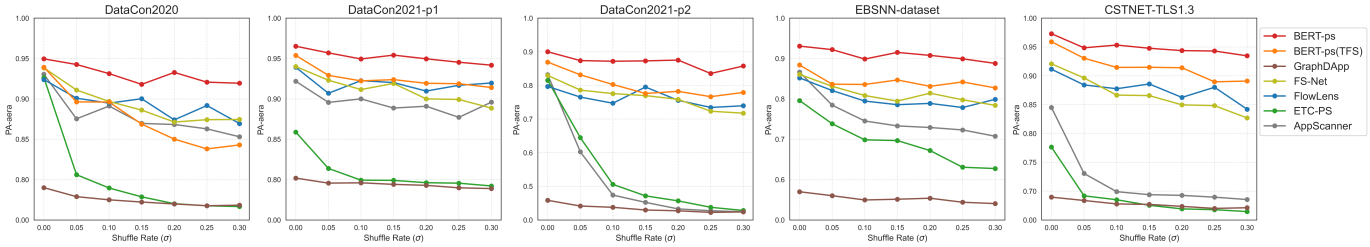
Method		Packet Loss						Packet Retransmission						Packet Disorder					
		0.05	0.1	0.15	0.2	0.25	0.3	0.05	0.1	0.15	0.2	0.25	0.3	0.1	0.2	0.3	0.4	0.5	0.6
AppScanner [3]		0.8930	0.8494	0.8315	0.8163	0.8051	0.7812	0.8753	0.8912	0.8696	0.8683	0.8628	0.8531	N/A					
ETC-PS [8]		0.5747	0.4360	0.3759	0.3331	0.3054	0.2739	0.8061	0.7124	0.6061	0.5089	0.4751	0.4620	0.7063	0.5427	0.4375	0.3246	0.2613	0.2293
FlowLens [5]		0.8567	0.7983	0.7549	0.6827	0.6292	0.5800	0.9009	0.8947	0.9002	0.8740	0.8918	0.8694	N/A					
FS-Net [9]		0.8784	0.8101	0.7732	0.7386	0.6990	0.6648	0.9110	0.8969	0.8862	0.8713	0.8742	0.8745	0.8958	0.8714	0.8411	0.8317	0.8208	0.8201
GraphDApp [10]		0.6090	0.5259	0.4481	0.3814	0.3900	0.3744	0.6084	0.5658	0.5338	0.5046	0.4762	0.4847	0.5155	0.3868	0.3686	0.3852	0.3724	0.3917
BERT-ps	TFS	0.8557	0.8051	0.7469	0.7059	0.6875	0.6294	0.8963	0.8961	0.8687	0.8502	0.8381	0.8430	0.8995	0.8876	0.8738	0.8609	0.8532	0.8229
	STF	**0.9129**	**0.8865**	**0.8665**	**0.8501**	**0.8270**	**0.7915**	**0.9426**	**0.9313**	**0.9180**	**0.9328**	**0.9207**	**0.9194**	**0.9297**	**0.9219**	**0.9188**	**0.9215**	**0.9283**	**0.9168**

Furthermore, Fig. 5 also presents variations in the PA-area metric across 3 types of network perturbation for all datasets.

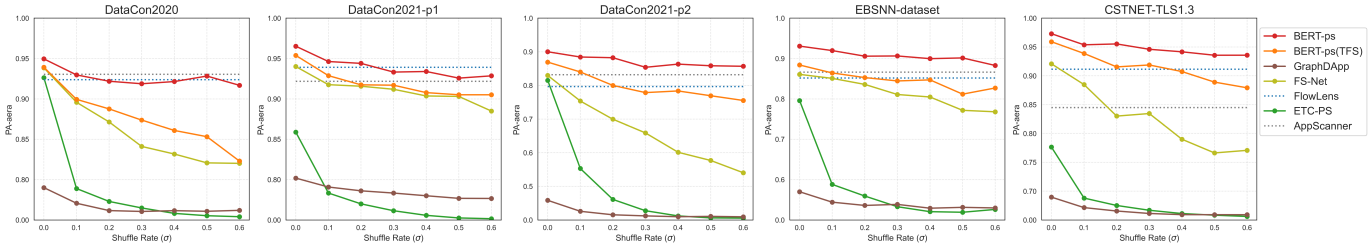
1) Stronger Robustness Than Baselines: As can be seen from Figure 4, the PA-curve of BERT-ps is farther to the right and higher than baselines under various types and levels of network noise. It indicates that our approach not only achieves higher accuracy but also exhibits stronger robustness. Most network flows are still correctly identified after perturbation. As illustrated in Table IV, the PA-area value of our approach decreases slower than existing methods as the network noise



(a) Packet Loss



(b) Packet Retransmission



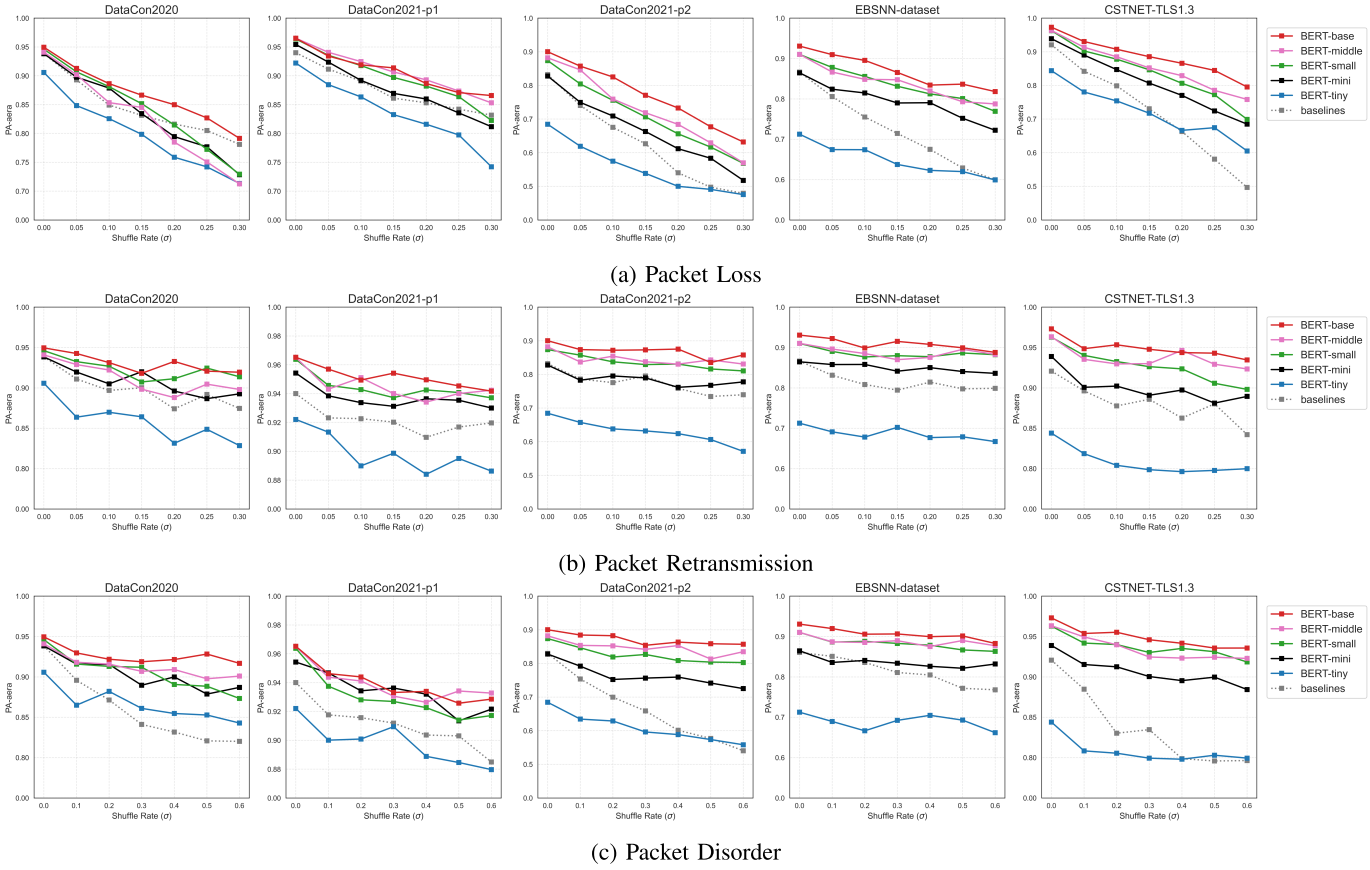
(c) Packet Disorder

Fig. 5. The PA-area under different types and intensities of network noise on five public datasets.

increases. This phenomenon can also be observed across all other datasets. Experimental results suggest that BERT-ps is more resilient to network noise. Existing methods are typically trained on labeled datasets, which tend to exhibit lower levels of noise compared to network traffic in real-world environments. As a result, models trained solely on clean labeled data often perform poorly when exposed to network traffic with noise. BERT-ps demonstrates stronger robustness because it is pre-trained on large-scale real-world network traffic with noise and obtains a stable and robust general knowledge of network traffic.

2) Pre-Training Can Enhance Robustness: On `DataCon2020`, although the fine-tuned model with pre-trained parameters and the model trained from scratch can achieve similar accuracy, they exhibit significant differences in robustness. As shown in Figure 4, the PA-curve of BERT-ps with SFT is shifted more to the right and upward compared to that of the TFS model. It indicates that the pre-trained model can increase the Δp of the sample's identification result during evaluation. Additionally, as demonstrated in Table IV, pre-training leads to a PA-area improvement up to 10% under common network noise. Besides `DataCon2020`, this phenomenon is also observed across all other datasets, which is illustrated in Fig. 5. Experimental results demonstrate that pre-training can enhance the model's robustness against network noises. The traffic data used for pre-training is collected from real-world network environments with various forms of network noise. During the pre-training process, the model can learn generalizable patterns that remain robust under diverse network noise conditions. This enables the proposed method to maintain a relatively stable PA-area under different network noises.

3) Robustness to Different Network Perturbation: We then analyze the impact of the three types of network noise on the model performance in more detail. As the packet loss rate increases (Fig. 5a), the model's performance gradually declines. However, the performance degradation of BERT-ps is notably less pronounced than without pre-training and other existing methods. Compared with packet re-transmission and disorder, packet loss has a stronger impact on the model's performance. For packet-length sequences, the high packet loss rate leads to a decrease in tokens, which will reduce the amount of information contained. Especially for "mouse flows" with fewer packets, packet loss will interfere with the identification results more severely. As illustrated in Fig. 5b, packet re-transmission has virtually no effect on our method, but it significantly degrades the performance of ETC-PS and GraphDApp. For this perturbation, some packet-length tokens will repeat consecutively but without disorder. Therefore, methods with less sensitivity to token duplication can better withstand packet re-transmission. Packet disorder also has a little impact on our approach. As shown in Table IV, the PA-area value of our approach is reduced by less than 2% with perturbation rates from 0.1 to 0.6. The features based on path signatures in ETC-PS are sensitive to the packet order information. Similarly, the RNN-based packet-length



(a) Packet Loss

(b) Packet Retransmission

(c) Packet Disorder

Fig. 6. The PA-aera under different types and intensities of network noise on 5 public datasets. The plot of `baselines` in the figure corresponds to the PA-area of the optimal baseline methods.

TABLE V
THE HYPER-PARAMETER SETTING OF BERT-PS

Model Scale	H	L	N_{head}	d_{FF}	**Params**
BERT-base	768	12	12	3072	88.1602M
BERT-middle	512	8	8	2048	27.1595M
BERT-small	512	4	8	2048	14.5500M
BERT-mini	256	4	4	1024	4.0652M
BERT-tiny	128	2	2	512	0.8348M

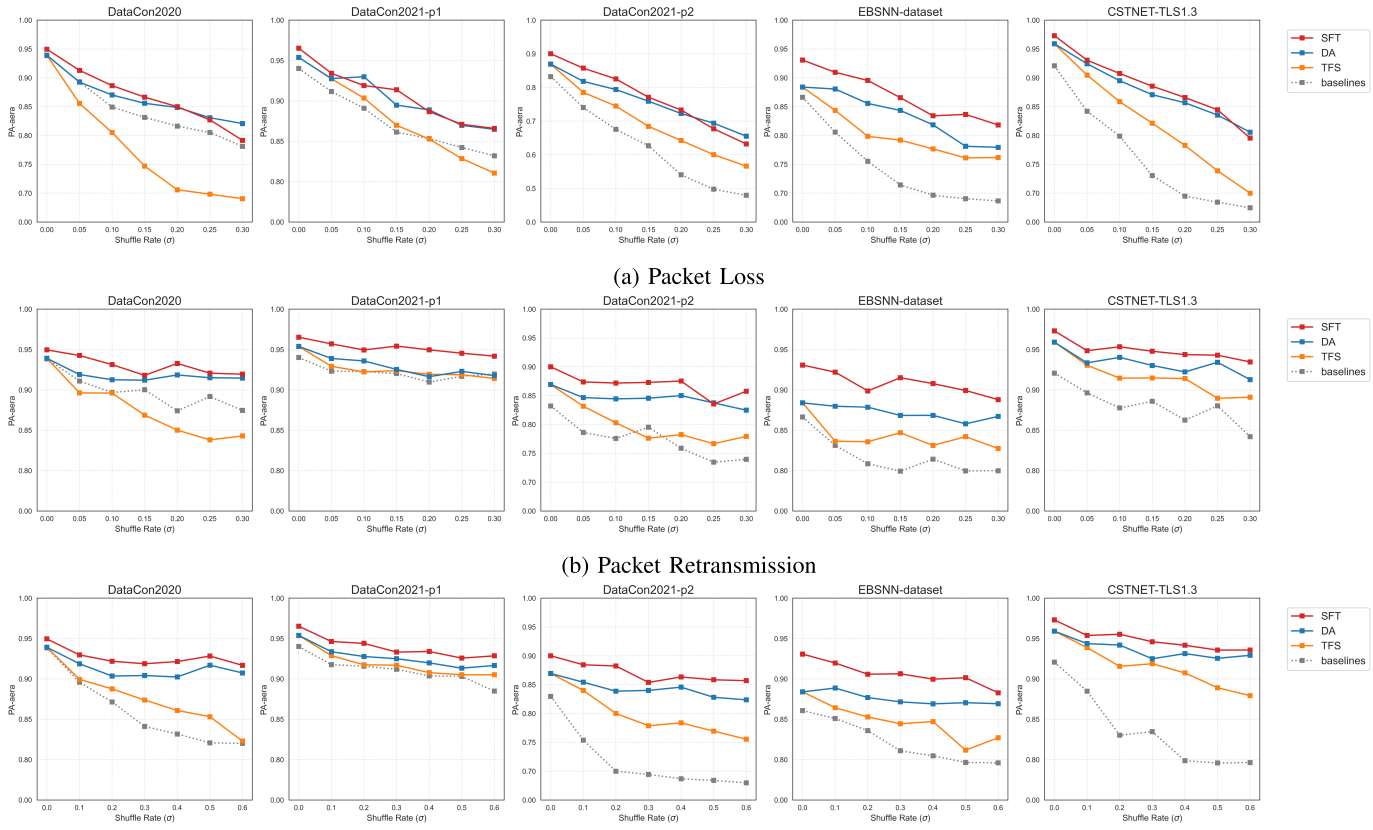
* H, L, N_{head}, and d_{FF} refer to hidden size, hidden layer number, attention head number, and intermediate size of feed forward layer.

representations employed in FS-Net also exhibit strong dependence on packet order. As for BERT-ps, the self-attention mechanism in the transformer encoder enables it to mitigate the impact of out-of-order noise, offering improved robustness to sequence irregularities. However, the performance of other methods (except for Appscanner and FlowLens) degrades more noticeably, which is also demonstrated in Fig. 5b. Additionally, it is worth noting that the features extracted by Appscanner and FlowLens are statistical features that are independent of packet order, making them unaffected by this perturbation. Consequently, as illustrated in Fig. 4, their PA-curves remain flat under packet disorder.

To analyze the impact of the pre-trained model's parameter scale on its robustness, we conducted tests on BERT-ps models with varying scales. We used the PA-area as the evaluation metric. The evaluated configurations include base, middle, small, mini, and tiny, with their hyper-parameters shown in Table V. Fig. 6 illustrates PA-area variations of pre-trained BERT-ps with different scales across five datasets under network noise. We also plotted the highest PA-area of baselines as a reference for comparison.

4) Larger Parameter Scales Enhance Robustness: As depicted in Fig. 6, the pre-trained model with larger scale exhibits stronger robustness compared to that of smaller-scale models. In general, as the size of model's parameters increases, the corresponding PA-area tends to be higher, especially on `DataCon2021-p2`, `EBSNN-dataset`, and `CSTNET-TLS1.3`. Notably, the tiny-scale pre-trained models typically demonstrate PA-area values lower than existing methods. It indicates that the advantages of pre-training become are evident only when model's parameters reach a certain scale. Specifically, as shown in Fig. 6, the pre-trained BERT-ps typically exhibit superior robustness compared to baselines when the parameter scale is larger than BERT-small.

5) Pre-Training Is Better Than Data Augmentation: In encrypted traffic analysis tasks, data augmentation is a common approach to enhance the model's robustness [19], [64], [65], [66], [67]. We compare the robustness improvement performance of two training strategies, data augmentation (DA) and pre-training with supervised fine-tuning (SFT). For data augmentation, we add network noise to the training data during supervised learning, including $\Upsilon_{\text{loss}}(0.2)$, $\Upsilon_{\text{retrans}}(0.2)$, and $\Upsilon_{\text{disorder}}(0.3)$.



(a) Packet Loss

(b) Packet Retransmission

(c) Packet Disorder

Fig. 7. The PA-area under different types and intensities of network noise on five public datasets using different training strategy for robustness improvement, including data augmentation (DA) and pre-training with supervised fine-tuning (SFT).

The experimental results are shown in Fig. 7, in which we also add the model trained from scratch (TFS) and the optimal baseline as a reference. As can be seen from the figure, the data augmentation strategy can improve the ability of the model to reduce noise to a certain extent under the three perturbations. However, its robustness is still weaker than that using the pre-training strategy. It indicates that data augmentation alone cannot substitute for the general knowledge learned from real-world network traffic data during pre-training, which illustrates the effectiveness of the pre-training on the improving model's robustness. While data augmentation can enhance model robustness by expanding the training distribution, it also significantly increases the computational overhead of model training due to the enlarged training set. In contrast, the pre-training strategy is more efficient and robust for improving model robustness, as it leverages prior knowledge by loading pre-trained parameters without increasing the training cost.

C. Time Efficiency

To assess real-world applicability, we evaluate the analysis latency of BERT-ps and existing methods, which is illustrated in Table VI. Experimental results indicate that BERT-ps can process approximately 1,000 network flows in 0.59 seconds. It shows that the latency of BERT-ps is higher than that of most existing baselines. This higher latency is primarily attributed to the model's large number of parameters. As can be seen from Table VI, the model analysis delay increases with the number of BERT-ps parameters increases.

TABLE VI

THE ANALYSIS LATENCY OF BASELINES AND BERT-PS WITH DIFFERENT PARAMETER SIZE

Method		FE (s/K)	MI (s/K)	AL (s/K)	Params
AppScanner		0.2126	0.0078	0.2204	0.1653M
ETC-PS		0.1690	0.0077	0.1767	0.1484M
FlowLens		0.0226	0.0113	0.0339	0.3162M
FS-Net		0.0673	0.0440	0.1113	2.7208M
GraphDApp		1.2922	0.0381	1.3303	0.2054M
BERT-ps	base	0.0715	0.5233	0.5948	88.1602M
	middle	0.0716	0.2122	0.2838	27.1595M
	small	0.0706	0.1223	0.1929	14.5500M
	mini	0.0706	0.0495	0.1201	4.0652M
	tiny	0.0702	0.0313	0.1015	0.8348M

* FE, MI refer to the time delay of feature extraction and model inference, respectively. AL refer to the analysis latency.

In addition, the analysis process comprises two components: feature extraction (FE) and model inference (MI). As illustrated in Table VI, their relative proportion varies significantly across models. For AppScanner, ETC-PS, and GraphDApp, feature extraction dominates the total latency, accounting for over 95% of the overall analysis time. In contrast, BERT-ps exhibits significantly lower feature extraction time (about 0.07s/K), representing only 12% of its total analysis time.

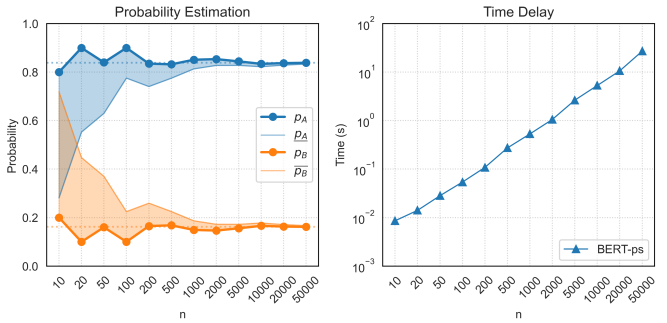


Fig. 8. The precision and efficiency of the model's decision probability estimation under different Monte Carlo sampling size (n).

Since the PA-curve is constructed based on the Monte Carlo method, we tested the precision and efficiency of the model's decision probability estimation under different sampling sizes (n). The target model we analyze is BERT-ps we proposed. We randomly selected a network flow sample with $p_A = 0.8384$ and $p_B = 0.1616$ for testing.

As shown in Fig. 8, the increasing samples can improve estimation precision but introduce a linear growth in time consumption. Notably, the estimation result of the Monte Carlo method begins to stabilize when the sample size exceeds 200. It suggests that the computational cost incurred by larger sample sizes does not scale proportionally with the gains in estimation accuracy. Therefore, an excessively large Monte Carlo sampling is not necessary for the construction of PA-curve. In practice, we set the Monte Carlo sample size to 1000, which not only ensures the estimation precision of the model's decision probability but also makes the estimation process an acceptable time delay (about 0.53s/sample). For large-scale datasets, down-sampling can be employed to further reduce computational overhead. Additionally, we also compute confidence intervals during the estimation process, which ensures the predicted probability differences $\Delta\hat{p}$ are not overestimated relative to the true values. As a result, the evaluation results of the model's robustness have a high confidence level.

VII. Discussion

The PA-curve and PA-area are reasonable quantification of model's robustness for encrypted traffic analysis. We discuss their advantages as follows:

- The model's robustness only makes sense when it accompanied by an acceptable accuracy. The PA-curve simultaneously captures both the accuracy and robustness of models, while the PA-area provides a quantitative description of the PA-curve. A model can achieve a higher PA-area value only when it exhibits both high accuracy and strong robustness.
- The PA-curve and PA-area depict model's robustness based on the distribution of sample's identification probability differences, providing a nuanced understanding of robustness. They account for difference robustness across samples, thereby measuring model's robustness based on the overall distribution of local robustness.
- The PA-curve and PA-area are model-agnostic and can be applied to assess the robustness of any identification model for encrypted traffic analysis, including both ML-based and DL-based models.

Through comprehensive robustness evaluations against traditional ML-based and DL-based approaches, we demonstrate that models with large-scale pre-trained parameters not only achieve high accuracy but also exhibit stronger robustness. Generally, the model's robustness improves with increasing scale of learnable parameters. Furthermore, pre-training enhances robustness compared to models trained from scratch. The pre-trained parameters have stabilized and exhibit strong robustness. The fine-tuning process usually does not significantly adjust pre-trained parameters, which ensures the model's robustness to downstream tasks.

However, there are still some limitations to be addressed. It is critical to acknowledge computational overhead as a primary limitation of the robustness metric. In PA-curve and PA-area, the computation of sample's identification probability differences is implemented using Monte Carlo methods. While increasing the number of samples can enhance the precision of probability estimation, it concurrently incurs significant computational cost with complexity $O(n)$. Consequently, future work may explore optimization techniques such as adaptive sampling, variance reduction, or quasi-Monte Carlo methods to improve computational efficiency while preserving estimation reliability. Additionally, although pre-trained models can bring high accuracy with strong robustness, they usually have large-scale parameters. The model size, inference latency, energy consumption, and dependence on dedicated GPU hardware limit their deployment in the real-world, especially on resource-constrained edge devices. Model compression techniques are expected to maintain robustness while improving deployment feasibility, including knowledge distillation, pruning, and quantization.

While our experiments focused on three common impacts of network noise (packet loss, retransmission, and disorder), we acknowledge that real-world network conditions may involve additional complexities such as mixed and non-uniform perturbations. Extending the robustness evaluation framework to accommodate these real-world scenarios is a promising direction for future work. Moreover, future research should explore architectural innovations to enhance the model's robustness, which BERT-ps does not consider. While larger pre-trained models generally exhibit stronger robustness, their pre-training process becomes increasingly challenging. Pre-training models with large-scale learnable parameters require not only vast amounts of training data and computational resources, but also appropriate optimization techniques to make pre-training loss converge. Consequently, the efficient pre-training of large-scale models for encrypted traffic analysis is also crucial for future work.

VIII. Conclusion

Considering the lack of reasonable robustness metrics for encrypted traffic analysis, in this paper, we design the PA-curve based on identification probability differences to illustrate the distribution of sample's decision stability. PA-curve can reflects model's accuracy and robustness, simultaneously. By computing the area under this curve, termed PA-area, we enable a quantitative assessment of model's robustness. Addi-

tionally, we propose a pre-trained packet-length sequence model based on the BERT architecture for encrypted traffic analysis and compare its performance with traditional ML-based and DL-based models trained from scratch. Besides accuracy, we analyzed the robustness of the pre-trained model and existing methods under three common network noise conditions. Experimental results demonstrate that the pre-trained model with large-scale parameters not only achieve higher accuracy, but also exhibit superior resilience to network noise compared to traditional ML-based and DL-based models trained from scratch. Therefore, investigating models with large-scale parameters for encrypted traffic analysis holds significant practical relevance.

APPENDIX

A. Clopper-Pearson Interval

The binomial proportion confidence interval is calculated from the outcome of a series of Bernoulli trials. When only the experiment number n and the success number k are known, we apply Clopper-Pearson method [57] to estimate the lower and upper bound of a success probability p. The Clopper-Pearson interval can be written as $\left(\inf S_{\leq}, \sup S_{\geq}\right)$ with

$$S_{\leq} \equiv \left\{p|\mathbb{P}\{X \leq k\} \leq \frac{\alpha}{2}\right\}, S_{\geq} \equiv \left\{p|\mathbb{P}\{X \leq k\} \geq \frac{\alpha}{2}\right\} \tag{9}$$

where $0 \leq k \leq n$ is the number of successes observed in the sample and X is the binomial random variable with n trials and probability of success p. The confidence level is $1-\alpha$.

For binomial distribution $X \sim \text{Bin}(n, p)$, its cumulative distribution function is defined as follows:

$$F_{\text{bin}}(k; n, p) = \sum_{i=0}^{k} \binom{n}{i} p^i (1-p)^{n-i}. \tag{10}$$

For beta distribution $X \sim B(n, p)$, its cumulative distribution function is defined as follows:

$$F_{\beta}(x; a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \int_0^x t^{a-1}(1-t)^{b-1} \mathrm{d}t \tag{11}$$

where $\Gamma(\cdot)$ is the gamma function. The beta distribution is the conjugate prior probability distribution for the binomial distribution. Thus, the cumulative distribution function of a beta distribution can be expressed in terms of the binomial distribution with

$$F_{\text{bin}}(k; n, p) = F_{\beta}(x = 1-p; a = n-k, b = k+1). \tag{12}$$

We can apply binomial distribution to estimate the lower bound $\underline{p}$ and the upper bound $\overline{p}$ of the success probability $p = \frac{k}{n}$. Moreover, they can also be presented in an alternate format that uses quantiles from the beta distribution, which is expressed as follows:

$$\begin{aligned} &F_{\text{bin}}(k; n, \overline{p}) = F_{\beta}(1-\overline{p}; n-k, k+1) = \frac{\alpha}{2}, \\ &F_{\text{bin}}(n-k; n, \underline{p}) = F_{\beta}\left(1-\underline{p}; k, n-k+1\right) = 1-\frac{\alpha}{2}. \end{aligned} \tag{13}$$

We define $F^{-1}_{\text{beta}}(\cdot)$ is the inverse cumulative distribution function (ICDF) of the beta distribution and

$$F^{-1}_{\beta}(x; a, b) = 1 - F^{-1}_{\beta}(1-x; b, a). \tag{14}$$

Finally, we estimate the lower and upper bound ($\underline{p}$ and $\overline{p}$) of a success probability as follows:

$$\begin{aligned} \overline{p} &= 1 - F^{-1}_{\beta}\left(\frac{\alpha}{2}; n-k, k+1\right) = F^{-1}_{\beta}\left(1-\frac{\alpha}{2}; k+1, n-k\right) \\ \underline{p} &= 1 - F^{-1}_{\beta}\left(1-\frac{\alpha}{2}; k, n-k+1\right) = F^{-1}_{\beta}\left(\frac{\alpha}{2}; n-k+1, k\right) \end{aligned} \tag{15}$$

B. Metrics About Accuracy

The fundamental goal of encrypted traffic analysis is to identify network flows correctly and avoiding misidentification. Metrics about accuracy include accuracy and macro-F_1.

The accuracy is the proportion of correctly identified samples to total samples, which can be expressed as follows:

$$Acc = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}(c_A^{(i)} = c^{(i)}) \tag{16}$$

where n is the total number of samples, $c^{(i)}$ and $c_A^{(i)}$ are respectively the true label and the prediction label of the i-th sample, and $\mathbb{I}(\cdot)$ is the indicator function.

In the binary case, there are four basic concepts: true positives (TP), false positives (FP), true negatives (TN) and false negatives (FN). F_1 score is defined as:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}. \tag{17}$$

where $P = \frac{TP}{TP+FP}$ is the precision and $R = \frac{TP}{TP+FN}$ is the recall rate. To generalize the binary case to multi-class, we can adopt the macro-F_1 assuming we have several one-vs-all (OvA) classifier of each individual class. Therefore, the macro-F_1 is the average of the F_1 scores of these classifier, which is defined as follow:

$$\text{macro-}F_1 = \frac{1}{|\mathcal{Y}|} \sum_{c \in \mathcal{Y}} F_1^{(c)} \tag{18}$$

where $F_1^{(c)}$ is the F_1-score of the classifier for c and $\mathcal{Y}$ is the label space of network flows.

REFERENCES

[1] B. Anderson and D. McGrew, "Identifying encrypted malware traffic with contextual flow data," in *Proc. ACM Workshop Artif. Intell. Secur.*, Oct. 2016, pp. 35–46.

[2] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy*, Mar. 2016, pp. 439–454.

[3] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.

[4] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Comput. Virol. Hacking Techn.*, vol. 14, no. 3, pp. 195–211, Aug. 2018.

[5] D. Barradas, N. Santos, L. Rodrigues, S. Signorello, F. M. V. Ramos, and A. Madeira, "FlowLens: Enabling efficient flow classification for ML-based network security applications," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.

[6] L. Yang et al., "Packet-sequence and byte-distribution is enough for real-time identification of application traffic," in *Proc. IEEE Intl Conf Parallel Distrib. Process. Appl., Big Data Cloud Comput., Sustain. Comput. Commun., Social Comput. Netw. (ISPA/BDCloud/SocialCom/SustainCom)*, Sep. 2021, pp. 1126–1134.

[7] L. Yang, S. Fu, Y. Wang, K. Liang, F. Mo, and B. Liu, "DEV-ETA: An interpretable detection framework for encrypted malicious traffic," *Comput. J.*, vol. 66, no. 5, pp. 1213–1227, May 2023.

[8] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, "Seeing traffic paths: Encrypted traffic classification with path signature features," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.

[9] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2019, pp. 1171–1179.

[10] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[11] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "EBSNN: Extended byte segment neural network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep. 2022.

[12] L. Yang, Y. Wang, L. Liu, J. Huang, and S. Fu, "ExpMD: An explainable framework for traffic identification based on multi-domain features," in *Proc. IEEE/ACM 32nd Int. Symp. Quality Service (IWQoS)*, Jun. 2024, pp. 1–10.

[13] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Industry-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.

[14] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[15] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[16] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.

[17] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–12.

[18] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1–15.

[19] R. Xie et al., "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-aware traffic augmentation," in *Proc. ACM Turing Award Celebration Conf. (China)*, Jul. 2023, pp. 131–132.

[20] D. Shan, F. Ren, P. Cheng, R. Shu, and C. Guo, "Observing and mitigating micro-burst traffic in data center networks," *IEEE/ACM Trans. Netw.*, vol. 28, no. 1, pp. 98–111, Feb. 2020.

[21] A. Yu, H. Yang, K. K. Nguyen, J. Zhang, and M. Cheriet, "Burst traffic scheduling for hybrid E/O switching DCN: An error feedback spiking neural network approach," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 882–893, Mar. 2021.

[22] N. Geng, T. Lan, V. Aggarwal, Y. Yang, and M. Xu, "A multi-agent reinforcement learning perspective on distributed traffic engineering," in *Proc. IEEE 28th Int. Conf. Netw. Protocols (ICNP)*, Oct. 2020, pp. 1–11.

[23] Y. Tian et al., "Traffic engineering in partially deployed segment routing over IPv6 network with deep reinforcement learning," *IEEE/ACM Trans. Netw.*, vol. 28, no. 4, pp. 1573–1586, Aug. 2020.

[24] M. Alfatafta, B. Alkhatib, A. Alquraan, and S. Al-Kiswany, "Toward a generic fault tolerance technique for partial network partitioning," in *Proc. 14th USENIX Symp. Operating Syst. Design Implement. (OSDI 20)*, 2020, pp. 351–368.

[25] Y. Yu et al., "Fault management in software-defined networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 1, pp. 349–392, 1st Quart., 2019.

[26] M. Reitblatt, N. Foster, J. Rexford, C. Schlesinger, and D. Walker, "Abstractions for network update," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 42, no. 4, pp. 323–334, Sep. 2012.

[27] N. Christensen, M. Glavind, S. Schmid, and J. Srba, "Latte: Improving the latency of transiently consistent network update schedules," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 48, no. 3, pp. 14–26, Mar. 2021.

[28] D. Han et al., "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2632–2647, Aug. 2021.

[29] M. Seo et al., "Heimdallr: Fingerprinting SD-WAN control-plane architecture via encrypted control traffic," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 949–963.

[30] Z. Song et al., "I^2 RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, pp. 1–14, 2024, doi: 10.1109/TDSC.2023.3242134. [Online]. Available: https://ieeexplore.ieee.org/document/10056861

[31] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, pp. 1–18, 2023, doi: 10.1109/TDSC.2023.3245411. [Online]. Available: https://ieeexplore.ieee.org/document/10036003

[32] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, May 2024, pp. 1608–1619.

[33] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "INSOMNIA: Towards concept-drift robustness in network intrusion detection," in *Proc. 14th ACM Workshop Artif. Intell. Secur.*, N. Carlini, A. Demontis, and Y. Chen, Eds., New York, NY, USA: ACM, Nov. 2021, pp. 111–122.

[34] Y. Li, B. Liang, and A. Tizghadam, "Robust online learning against malicious manipulation with application to network flow classification," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–10.

[35] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "MANDA: On adversarial example detection for network intrusion detection system," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–10.

[36] N. Wang, Y. Chen, Y. Xiao, Y. Hu, W. Lou, and Y. T. Hou, "MANDA: On adversarial example detection for network intrusion detection system," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1139–1153, Mar. 2023.

[37] X. Xiao et al., "RBLJAN: Robust byte-label joint attention network for network traffic classification," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 3, pp. 2161–2178, May 2025.

[38] S. Feghhi and D. J. Leith, "A web traffic analysis attack using only timing information," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 8, pp. 1747–1759, Aug. 2016.

[39] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.

[40] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.

[41] J. Li et al., "Packet-level open-world app fingerprinting on wireless traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–18.

[42] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.

[43] J. Qu et al., "An input-agnostic hierarchical deep learning framework for traffic fingerprinting," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 589–606.

[44] K. Wang et al., "BARS: Local robustness certification for deep learning based traffic analysis systems," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.

[45] F. Wei, H. Li, Z. Zhao, and H. Hu, "xNIDS: Explaining deep learning-based network intrusion detection systems for active intrusion responses," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 4337–4354.

[46] M. Korczynski and A. Duda, "Markov chain fingerprinting to classify encrypted traffic," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2014, pp. 781–789.

[47] M. Shen, M. Wei, L. Zhu, M. Wang, and F. Li, "Certificate-aware encrypted traffic classification using second-order Markov chain," in *Proc. IEEE/ACM 24th Int. Symp. Quality Service (IWQoS)*, Jun. 2016, pp. 1–10.

[48] M. Shen, M. Wei, L. Zhu, and M. Wang, "Classification of encrypted traffic with second-order Markov chains and application attribute bigrams," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 8, pp. 1830–1843, Aug. 2017.

[49] W. Pan, G. Cheng, and Y. Tang, "WENC: HTTPS encrypted traffic classification using weighted ensemble learning and Markov chain," in *Proc. IEEE Trustcom/BigDataSE/ICESS*, Aug. 2017, pp. 50–57.

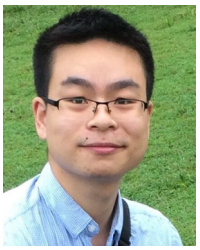
[50] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, "MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints," in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.

[51] L. Yang et al., "CADE: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Secur. Symp.*, Vancouver, BC, Canada, 2021, pp. 2327–2344.

[52] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.
[53] Z. Fu et al., "Encrypted malware traffic detection via graph-based network analysis," in *Proc. 25th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2022, pp. 495–509.
[54] A. Wieland and C. M. Wallenburg, "Dealing with supply chain risks: Linking risk management practices and strategies to performance," *Int. J. Phys. Distribution Logistics Manage.*, vol. 42, no. 10, pp. 887–905, Nov. 2012.
[55] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 1310–1320.
[56] L. D. Brown, T. T. Cai, and A. DasGupta, "Interval estimation for a binomial proportion," *Stat. Sci.*, vol. 16, no. 2, pp. 101–133, May 2001.
[57] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, Dec. 1934.
[58] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
[59] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," 2018. [Online]. Available: https://cdn.openai.com/research-covers/languageunsupervised/language_understanding_paper.pdf
[60] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
[61] DataCon-Community. (2020). *Datacon2020–Encrypted Malicious Traffic Dataset*. [Online]. Available: https://datacon.qianxin.com/opendata/openpage?resourcesId=6
[62] DataCon-Community. (2021). *Datacon2021–Encrypted Proxy Traffic Dataset*. [Online]. Available: https://datacon.qianxin.com/opendata/openpage?resourcesId=10
[63] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu. (2022). *CSTNet-TLS 1.3 Dataset*. [Online]. Available: https://drive.google.com/drive/folders/1BUo5TMRuXNvTqNYy0RLeHk4l4Q3BuzS%k
[64] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-FlowPic augmentations," in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 647–654.
[65] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1035–1049.
[66] X. Zhao et al., "Towards fine-grained webpage fingerprinting at scale," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 423–436.
[67] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, "Robust detection of malicious encrypted traffic via contrastive learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4228–4242, 2025.

Luming Yang received the Ph.D. degree in cyberspace security from the National University of Defense Technology in 2025. He is currently an Assistant Researcher with the Academy of Military Science. His research interests include encrypted traffic analysis, intrusion detection, network security, and machine learning.

Lin Liu received the Ph.D. degree in computer science from the National University of Defense Technology, Changsha, China, in 2020. Currently, he is an Associate Professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include applied cryptography, privacy-preserving machine learning, and encrypted network traffic detection.



Jun-Jie Huang (Member, IEEE) received the B.Eng. degree (Hons.) in electronic engineering and the M.Phil. degree in electronic and information engineering from The Hong Kong Polytechnic University, Hong Kong, China, in 2013 and 2015, respectively, and the Ph.D. degree from Imperial College London (ICL), London, U.K., in 2019. From 2019 to 2021, he was a Post-Doctoral Researcher with the Communications and Signal Processing (CSP) Group, Electrical and Electronic Engineering Department, ICL. He is currently an Associate Professor with the College of Computer Science and Technology, National University of Defense Technology (NUDT), Changsha, China. His research interests include the areas of model-based deep learning, computer vision, and signal processing.

Jiangyong Shi was born in 1990. He received the Ph.D. degree. He is currently an Associate Professor. His research interests include networks and system security.

Shaojing Fu received the Ph.D. degree in applied mathematics from the National University of Defense Technology in 2010. He is currently a Professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include cryptography theory and information security.

Yongjun Wang received the Ph.D. degree in computer architecture from the National University of Defense Technology in 1998. He is currently a Professor with the College of Computer Science and Technology, National University of Defense Technology. His research interests include network security and system security.

Jinshu Su (Senior Member, IEEE) received the B.S. degree in mathematics from Nankai University, Tianjin, in 1985, and the M.S. and Ph.D. degrees in computer science from the National University of Defense Technology, Changsha, in 1988 and 2000, respectively. He is currently a Professor with the Academy of Military Science. His research interests include internet architecture, network security, and AI for networking.